

Вы движетесь в правильную сторону. То, что вы сейчас делаете интуитивно, называется **моделированием данных и информационной архитектурой организации**. Проблема не в том, что вы «неправильно выделяете сущности», а в том, что вы пытаетесь одновременно решить несколько разных задач:

- описать процессы;
- определить бизнес-сущности;
- собрать атрибуты;
- разобраться с дублированием;
- определить источники данных;
- спроектировать таблицы базы данных;
- продумать интеграцию с EDMS, Excel и будущей ERP.

Эти задачи необходимо выполнять последовательно.

1. Не начинайте сразу с таблиц базы данных

Архитектура данных строится на трех уровнях.

Уровень 1. Концептуальная модель

На этом уровне вы определяете, **какие объекты существуют в деятельности NURA**:

- человек;
- подразделение;
- проект;
- заявка на грант;
- грантовый конкурс;
- договор;
- бюджет;
- закупочная заявка;
- платеж;
- отчет;
- документ.

Пока без технических полей и типов данных.

Уровень 2. Логическая модель

Здесь вы определяете:

- атрибуты объектов;
- связи между ними;
- обязательность связей;

- правила;
- историю изменений;
- уникальные идентификаторы.

Например:

Один человек может участвовать в нескольких проектах.
В одном проекте может участвовать несколько человек.
В каждом проекте один из участников может выполнять роль PI.

Уровень 3. Физическая модель

Только после этого проектируются:

- таблицы;
- первичные и внешние ключи;
- индексы;
- справочники;
- API;
- механизмы интеграции;
- журналирование изменений.

Сейчас вам нужно работать преимущественно на первых двух уровнях.

2. Главное правило моделирования данных

При анализе любого поля задавайте вопрос:

Что это за факт, к какому объекту он относится, в каком контексте и на какую дату он действителен?

Например, поле «Школа PI» может означать три разных факта:

1. школа, в которой человек работает сейчас;
2. школа, в которой человек работал на момент подачи заявки;
3. школа, которая официально администрирует проект.

Это не одно и то же.

Если просто создать в таблице проекта колонку `Школа PI`, позднее возникнет проблема: человек перейдет в другую школу — и станет непонятно, должна ли школа измениться во всех старых проектах.

Поэтому при моделировании всегда учитываются три вещи:

- **идентичность объекта;**

- контекст связи;
- время действия данных.

3. В вашем примере PI — это не самостоятельная сущность

Вы написали:

1. PI
- имя
 - телефон
 - должность

Здесь смешаны **человек** и его **роль в проекте**.

PI — Principal Investigator — это не вид человека. Это роль, которую человек выполняет в определенном проекте.

Правильнее разделить модель следующим образом.

Сущность «Человек»

Атрибут	Содержание
Person ID	внутренний уникальный идентификатор
Фамилия	фамилия
Имя	имя
Отчество	при наличии
Email	контактный адрес
Телефон	контактный номер
Статус	активен, уволен, внешний исследователь и т. д.

Не следует использовать ФИО как идентификатор. У двух людей могут совпадать имена, а фамилия может измениться.

Сущность «Организационное подразделение»

Вместо отдельных жестко заданных таблиц «Школа», «Факультет», «Департамент» можно использовать общую сущность:

Атрибут	Содержание
Unit ID	уникальный идентификатор
Наименование	School of Engineering, NURA и т. д.
Тип подразделения	школа, департамент, центр, офис
Вышестоящее подразделение	ссылка на родительское подразделение
Дата начала действия	дата создания
Дата окончания действия	при закрытии или реорганизации
Статус	активно, закрыто, реорганизовано

Так вы сможете описать иерархию:

```

Nazarbayev University
├─ School of Engineering and Digital Sciences
│   └─ Department A
│       └─ Department B

```

Для этого используется связь подразделения с самим собой:

```

Организационное подразделение
├─ parent_unit_id

```

Сущность «Работа или аффилиация человека»

Связь человека со школой нельзя просто записать в поле `school_id` человека, потому что:

- человек может перейти в другое подразделение;
- иметь несколько аффилиаций;
- работать в школе и одновременно в исследовательском центре;
- менять должность.

Поэтому нужна отдельная сущность:

Аффилиация человека

Атрибут	Содержание
Affiliation ID	идентификатор записи

Атрибут	Содержание
Person ID	человек
Unit ID	подразделение
Должность	профессор, исследователь и т. д.
Дата начала	начало работы
Дата окончания	окончание работы
Основное место работы	да/нет
Статус	активно/завершено

Получается:

Человек

1

|

└──< Аффилиация >──1 Организационное подразделение

Один человек может иметь несколько записей об аффилиации во времени.

4. Как правильно связать человека с проектом

Не нужно записывать `PI ID` непосредственно в таблице проекта, если система должна поддерживать разные роли участников.

Следует создать сущность «Участие в проекте».

Проект

Атрибут	Содержание
Project ID	внутренний идентификатор
Наименование проекта	официальное название
Краткое наименование	при необходимости
Дата начала	начало проекта

Атрибут	Содержание
Дата окончания	окончание проекта
Статус	планируется, активен, завершен и т. д.
Администрирующее подразделение	ответственная школа или подразделение

Участие в проекте

Атрибут	Содержание
Project ID	проект
Person ID	человек
Роль	PI, Co-PI, researcher, project manager
Дата начала участия	начало участия
Дата окончания участия	окончание участия
Доля участия	при необходимости
Статус	активен/завершил участие

Связь выглядит так:



Это позволяет:

- одному человеку быть PI в нескольких проектах;
- одному человеку быть PI в одном проекте и исследователем в другом;
- хранить Co-PI;
- заменять PI;
- сохранять историю;
- учитывать внешних участников.

5. Дублирующиеся номера не всегда являются дублирующимися данными

Ваш пример с EDMS и реестром закупок очень важен.

В EDMS появляется один номер, а отдел закупок присваивает другой. Ошибкой будет выбрать один из них как «правильный» и уничтожить второй.

Сначала нужно определить, относятся ли номера к одному объекту.

Вариант 1. Это два номера одной закупочной заявки

Тогда существует одна сущность:

Закупочная заявка

Атрибут	Содержание
Procurement Request ID	внутренний идентификатор ERP
Project ID	проект
Инициатор	сотрудник
Дата создания	дата
Предмет закупки	описание
Сумма	сумма
Статус	статус

И отдельная таблица идентификаторов:

Внешние идентификаторы закупочной заявки

Request ID	Система	Тип номера	Значение
845	EDMS	Номер документа	EDMS-2026-00125
845	Реестр закупок	Регистрационный номер	PR-4568

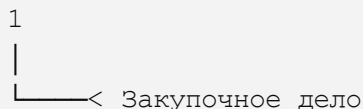
Внутри ERP заявка имеет стабильный идентификатор, например 845 . Номера EDMS и отдела закупок являются внешними идентификаторами этой заявки.

Вариант 2. Это разные бизнес-объекты

Возможно, в EDMS создается **заявка инициатора**, а отдел закупок позднее создает на ее основе **закупочное дело**.

Тогда модель должна быть другой:

Закупочная заявка



Причем возможны ситуации:

- одна заявка превращается в несколько закупочных процедур;
- несколько заявок объединяются в одну закупку;
- часть заявки отменяется;
- заявка возвращается на доработку;
- закупка делится на лоты.

Поэтому нельзя принимать решение только на основании того, что записи имеют похожие названия. Нужно выяснить, имеют ли они одинаковый жизненный цикл.

6. Как отличить сущность от атрибута

Используйте простой тест.

Это сущность, если объект:

- имеет собственный жизненный цикл;
- может существовать независимо;
- имеет собственный идентификатор;
- участвует в нескольких связях;
- имеет историю;
- по нему формируются отчеты;
- создается, меняется, закрывается или аннулируется.

Например, «проект» — сущность.

Это атрибут, если значение:

- только описывает объект;
- не имеет собственного жизненного цикла;
- обычно не используется независимо.

Например, «краткое наименование проекта» — атрибут проекта.

Это справочник, если значение выбирается из контролируемого списка

Например:

- тип гранта;
- статус проекта;
- роль участника;
- вид договора;
- валюта;
- тип подразделения;
- категория расходов.

Это связь, если значение описывает отношение объектов

Например:

- человек работает в школе;
- человек участвует в проекте;
- проект финансируется грантом;
- заявка относится к проекту;
- платеж относится к договору.

7. Не пытайтесь сразу описать все данные NURA

Лучше разделить информационную архитектуру на домены.

Для NURA предварительно можно выделить следующие области.

1. Организационная структура и люди

- человек;
- сотрудник;
- внешнее лицо;
- подразделение;
- должность;
- аффилиация;
- роль пользователя;
- контактные данные.

2. Грантовые конкурсы и заявки

- финансирующая организация;
- грантовая программа;
- конкурс;

- заявка;
- заявитель;
- участник заявки;
- решение;
- рецензия;
- документ заявки.

3. Научные проекты

- проект;
- участник проекта;
- роль участника;
- этап проекта;
- результат;
- КРІ;
- отчет;
- изменение проекта.

4. Договоры и финансирование

- грантовое соглашение;
- договор;
- дополнительное соглашение;
- источник финансирования;
- бюджет;
- бюджетная статья;
- финансовый год;
- транш.

5. Закупки и расходы

- потребность;
- закупочная заявка;
- закупочное дело;
- лот;
- поставщик;
- договор закупки;
- счет;
- платеж;
- приемка.

6. Документы и согласования

- документ;

- тип документа;
- версия документа;
- согласование;
- участник согласования;
- решение;
- статус;
- номер во внешней системе.

При этом EDMS, Excel и будущая ERP — это **не бизнес-сущности**, а системы, в которых хранятся сведения о сущностях.

8. Методология работы

Для каждого выбранного процесса проходите одинаковые этапы.

Этап 1. Определить границы процесса

Например:

От возникновения потребности в закупке до заключения договора и оплаты.

Нельзя анализировать просто «все закупки». Нужно определить начало и конец.

Этап 2. Описать шаги процесса

Для каждого шага фиксируйте:

- кто выполняет действие;
- что является входом;
- что создается;
- какое решение принимается;
- какие данные появляются;
- в какой системе они фиксируются;
- какой документ формируется;
- какой статус меняется.

Пример:

Шаг	Исполнитель	Действие	Создаваемые данные	Система
1	PI	Формирует потребность	описание, сумма, проект	Excel
2	Администратор	Проверяет бюджет	статья бюджета, остаток	Excel

Шаг	Исполнитель	Действие	Создаваемые данные	Система
3	Инициатор	Направляет заявку	номер документа	EDMS
4	Закупщик	Регистрирует заявку	номер реестра	Excel

Так вы увидите, почему данные дублируются и где возникают разрывы.

Этап 3. Выделить бизнес-сущности

Из процесса выписывайте существительные, но не превращайте автоматически каждое существительное в таблицу.

Например:

- PI → человек + роль;
- школа → организационное подразделение;
- бюджет проекта → бюджет;
- заявка на закуп → закупочная заявка;
- номер EDMS → внешний идентификатор;
- согласование → отдельный процесс или запись согласования.

Этап 4. Описать жизненный цикл каждой сущности

Например, закупочная заявка:

Черновик
→ Подана
→ На проверке
→ Возвращена
→ Согласована
→ Передана в закупки
→ В работе
→ Завершена / Отменена

Статус должен описывать состояние конкретного объекта, а не просто этап всего процесса.

Этап 5. Определить связи и их кратность

Для каждой пары объектов задавайте вопросы:

- Может ли один проект иметь несколько заявок?
- Может ли одна заявка относиться к нескольким проектам?
- Может ли один человек работать в нескольких подразделениях?

- Может ли проект иметь нескольких PI одновременно?
- Может ли один договор охватывать несколько проектов?

Возможные типы связей:

- один к одному;
- один ко многим;
- многие ко многим.

Связь «многие ко многим» почти всегда оформляется отдельной сущностью.

Этап 6. Создать словарь данных

Для каждого атрибута следует фиксировать не только его название.

Минимальная структура словаря:

Поле	Что фиксируется
Домен	проекты, закупки, договоры
Сущность	закупочная заявка
Атрибут	дата подачи
Определение	дата официальной регистрации заявки
Тип данных	дата
Обязательность	да/нет
Допустимые значения	при наличии
Источник	EDMS
Владелец данных	отдел закупок
Кто вводит	инициатор
Правило изменения	после регистрации не изменяется
История изменений	требуется/не требуется
Конфиденциальность	обычные/персональные/финансовые данные
Проверка качества	не может быть раньше даты создания

Это один из основных документов для будущей ERP.

9. Для каждого значения должен быть определен источник истины

Нужно создать матрицу источников данных.

Например:

Данные	Главный источник	Ответственный
ФИО сотрудника	HR-система	HR
Текущее подразделение	HR-система	HR
Название проекта	ERP NURA	Project Administration
Номер документа EDMS	EDMS	делопроизводство
Номер закупочного дела	система закупок	отдел закупок
Сумма утвержденного бюджета	финансовая система	Finance
Фактические расходы	ERP/финансовая система	Finance

Главный источник не означает, что данные существуют только там. Они могут копироваться в другие системы, но изменяться должны в одном определенном месте.

10. Базовые принципы правильной архитектуры

Один факт хранится в одном месте

Название школы не нужно многократно вводить вручную в проектах, договорах и закупках. В этих объектах хранится ссылка на подразделение.

Каждый объект имеет внутренний идентификатор

Например:

```
project_id = 1025
```

Название проекта может измениться, а идентификатор остается прежним.

Внешние номера хранятся отдельно

Номер EDMS, номер договора, номер конкурса и номер закупочного реестра могут относиться к одному объекту, но выполняют разные функции.

История не должна уничтожаться

Если PI сменил школу, старые записи не должны автоматически переписываться.

Роль отделяется от человека

PI, руководитель подразделения, заявитель, согласующий и владелец бюджета — это роли в определенном контексте.

Справочники не вводятся свободным текстом

Нельзя допускать одновременно:

- School of Engineering;
- SEDS;
- Школа инженерии;
- School of Engineering and Digital Sciences.

Это должно быть одно подразделение с одним `unit_id`, официальным названием и, при необходимости, альтернативными названиями.

11. Минимальное ядро модели NURA

На первом этапе я бы строил ядро вокруг следующих сущностей:

```
PERSON
ORGANIZATIONAL_UNIT
PERSON_AFFILIATION

FUNDING_ORGANIZATION
GRANT_PROGRAM
GRANT_CALL
GRANT_APPLICATION
APPLICATION_PARTICIPANT

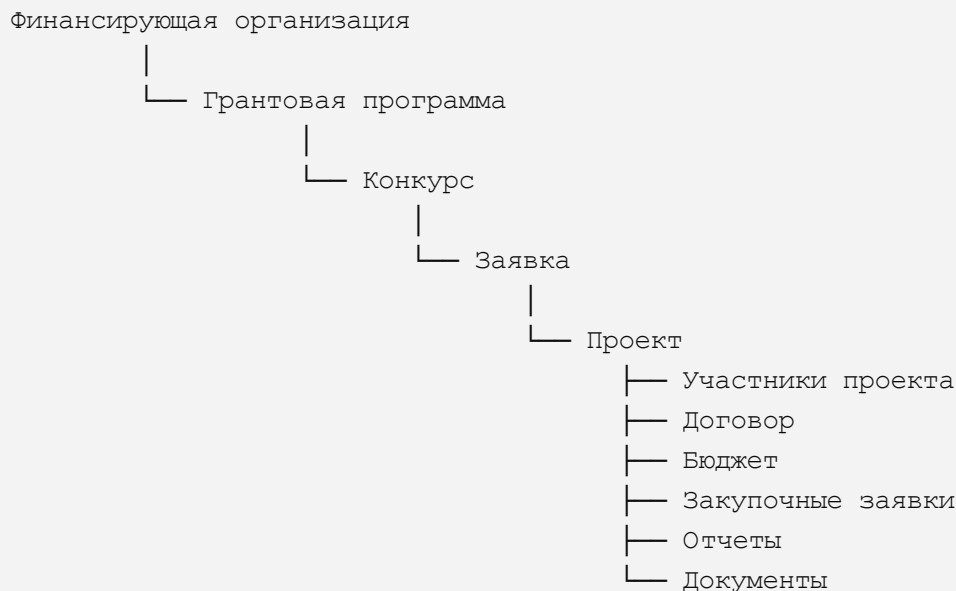
PROJECT
PROJECT_PARTICIPANT
PROJECT_ORGANIZATIONAL_UNIT

AGREEMENT
BUDGET
BUDGET_LINE
```

PROCUREMENT_REQUEST
PROCUREMENT_CASE

DOCUMENT
EXTERNAL_IDENTIFIER
WORKFLOW_ACTION

Пример основных связей:



12. С чего начать практически

Не пытайтесь сейчас построить полную модель NURA. Возьмите один сквозной процесс, например:

От объявления грантового конкурса до создания проекта и первой закупочной заявки.

Для него подготовьте шесть документов:

1. схему процесса;
2. перечень сущностей;
3. карточки сущностей;
4. схему связей;
5. словарь данных;
6. матрицу источников и владельцев данных.

Только после согласования этих материалов следует проектировать таблицы ERP.

Самая важная корректировка вашего подхода: **выделяйте не просто поля, а устойчивые бизнес-объекты, их роли, связи, жизненные циклы и историю.**

Именно это превращает фрагментарный перечень данных в системную архитектуру.

